СРС №1

Event Map App

Сервис поиска и публикации локальных мероприятий

Архитектура, REST API, база данных и безопасность

Дисциплина: Разработка веб-приложений

Студент: Фамилия Имя, группа ____

Преподаватель: Самат Илияс Мырзабекулы

Алматы, 2026

Аннотация

Документ описывает MVP Event Map App: предметную область, требования, архитектуру трёх сервисов, модель данных, контракт REST API, аутентификацию и авторизацию, единый формат ошибок и анализ угроз OWASP API Security Top 10. Материалы соответствуют ПР1–ПР4 и используются как пояснительная записка к презентации СРС №1.

Содержание

1. Введение
2. Требования и пользовательские истории
3. Выбор технологий
4. Архитектура приложения
5. Структура репозитория и конфигурация
6. Модель данных
7. Миграции, ограничения и индексы
8. Контракт REST API
9. Валидация и единый формат ошибок
10. Пагинация и фильтрация
11. Аутентификация
12. Авторизация и роли
13. Безопасность API
14. Демонстрация работы
15. Статус проекта и план
16. Заключение
17. Список использованных источников
Приложение А. Команды запуска и проверки

1. Введение

1.1. Предметная область

Event Map App помогает жителям города находить локальные мероприятия на карте: концерты, лекции, выставки и встречи. Пользователь может фильтровать события по категории и дате, отмечать «пойду» или «интересно». Организаторы публикуют мероприятия и видят отклик аудитории.

Ключевая ценность продукта — быстрый поиск событий рядом с собой и простой способ публикации без сложной инфраструктуры. Для MVP достаточно трёх независимых сервисов и единого контракта API.

1.2. Цель и задачи

Цель работы — спроектировать и реализовать MVP сервиса, в котором:

- пользователь находит события на карте и фильтрует их;
- организатор публикует событие и видит статистику отметок;
- аутентификация и роли едины для всех сервисов;
- контракт API зафиксирован в OpenAPI, а ошибки имеют единый формат.

Для достижения цели решаются задачи:

- сформулировать бизнес-задачу и пользовательские истории;
- выбрать стек и спроектировать архитектуру из трёх сервисов;
- описать модель данных, миграции, ограничения и индексы;
- реализовать REST API, валидацию, пагинацию и обработку ошибок;
- добавить JWT-аутентификацию, RBAC и проверку владельца объекта;
- проверить сценарии доступа автотестами и проанализировать угрозы OWASP.

1.3. Целевая аудитория

Основные пользователи — жители города, которые ищут локальные события, и небольшие организаторы: клубы, лекторы, образовательные проекты и локальные сообщества. Отдельная роль admin нужна для модерации и просмотра пользователей.

2. Требования и пользовательские истории

Требования сформулированы в формате пользовательских историй. Для каждой истории задан критерий приёмки, который проверяется вручную или автотестами.

ID	Пользовательская история	Критерий приёмки
US-1	Просмотр мероприятий на карте	Пользователь видит маркеры событий в радиусе 10 км
US-2	Фильтрация по категории и дате	В списке остаются только подходящие события
US-3	Публикация мероприятия	Organizer заполняет форму, событие видно всем
US-4	Отметка участия	Отметка сохраняется, счётчик увеличивается на 1
US-5	Статистика организатора	«Пойду» и «интересно» показаны отдельно
US-6	Регистрация и выбор роли	Создаётся user или organizer; admin — отдельно

Таблица 1 — Пользовательские истории MVP

Definition of Done для MVP:

- все шесть историй покрыты API и проверены вручную;

• для успешных и ошибочных сценариев возвращаются корректные HTTP-коды;
• изменяющие запросы защищены JWT, роли и проверкой владельца;
• контракт API описан в OpenAPI, примеры запросов лежат в docs/examples.http;
• автотесты users-service, events-service и attendance-service проходят.

3. Выбор технологий

Для реализации рассматривались Flask, Django и FastAPI. Сравнение проводилось по наличию ORM и миграций, готовой аутентификации, админ-панели, скорости разработки MVP и поддержке async.

Критерий	Flask	Django	FastAPI
ORM и миграции	нет из коробки	ORM + миграции	SQLAlchemy / Alembic
Auth и admin	собрать вручную	готовые auth + admin	собрать вручную
Скорость MVP	высокая	средняя	высокая
Async	нет	нет	да
Решение	events, attendance	users-service	не выбран для MVP

Таблица 2 — Сравнение вариантов стека

Для events-service и attendance-service выбран Flask: обоим сервисам нужен компактный REST/CRUD без сложной админки. Для users-service выбран Django: он даёт готовую модель пользователя, хеширование паролей, миграции и админ-панель. FastAPI не выбран для MVP, потому что async не является критичным, а готовую аутентификацию и админку пришлось бы собирать отдельно.

Итоговый стек: Python 3.12, Django 5.0.7, Flask 3.0.3, SQLAlchemy 2, Alembic, SQLite локально и PostgreSQL через DATABASE_URL, PyJWT, argon2-cffi, OpenAPI 3.0.

4. Архитектура приложения

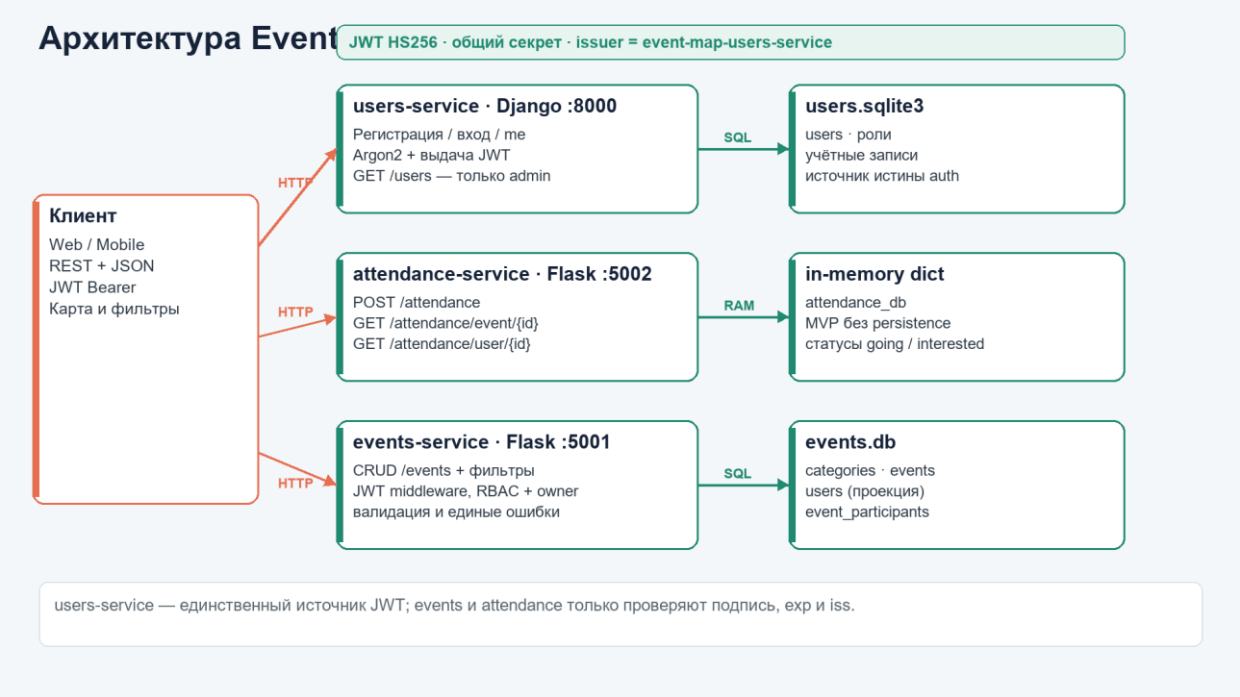


Рисунок 1 — Архитектура Event Map App

Сервис	Ответственность	Данные	Порт
users-service	Регистрация, вход, JWT, роли, профиль	users, роли	8000
events-service	CRUD мероприятий, фильтры, владелец	events, categories, users-проекция	5001
attendance-service	Отметки «пойду» / «интересно»	attendance (in-memory MVP)	5002

Таблица 3 — Сервисы и зоны ответственности

Основной сценарий взаимодействия:

1. Клиент регистрируется или входит через users-service и получает JWT.
2. Клиент передаёт токен в заголовке Authorization: Bearer <token>.
3. Events и attendance проверяют подпись, exp и iss локально общим секретом.
4. events-service дополнительно проверяет роль и владельца события.
5. Ошибки всех сервисов приводятся к единому JSON-формату.

users-service — источник истины для аутентификации. В events-service есть отдельная таблица users как проекция для внешних ключей и регистраций; автоматическая синхронизация этой проекции с users-service пока не реализована и вынесена в план PR5.

Внутри events-service используется слоистая структура: маршрут → валидация → сервис → репозиторий → база данных. Граница транзакции находится в database.db_session: commit при успехе и rollback при ошибке.

5. Структура репозитория и конфигурация

```
event-map-app/
├── users-service/        Django: auth, JWT, admin
├── events-service/       Flask: events, Alembic, OpenAPI
├── attendance-service/   Flask: отметки участия
├── docs/                 требования, auth, security, CPC1
└── README.md             запуск, тесты, PR1–PR4
```

Секреты не хранятся в коде. В репозитории есть только .env.example с шаблоном переменных; реальные значения задаются в .env и не коммитятся. Для production DEBUG выключен.

Переменная	Назначение	Пример
JWT_SECRET	общий секрет подписи JWT	change-me-in-production
JWT_ALGORITHM	алгоритм подписи	HS256
JWT_ISSUER	издатель токена	event-map-users-service
JWT_EXPIRES_MINUTES	время жизни access-токена	60
DJANGO_SECRET_KEY	секрет Django	change-me
DJANGO_DEBUG	режим отладки	0
DATABASE_URL	подключение к БД	sqlite:///events.db
FLASK_DEBUG	режим отладки Flask	0

Таблица 4 — Основные переменные окружения

6. Модель данных

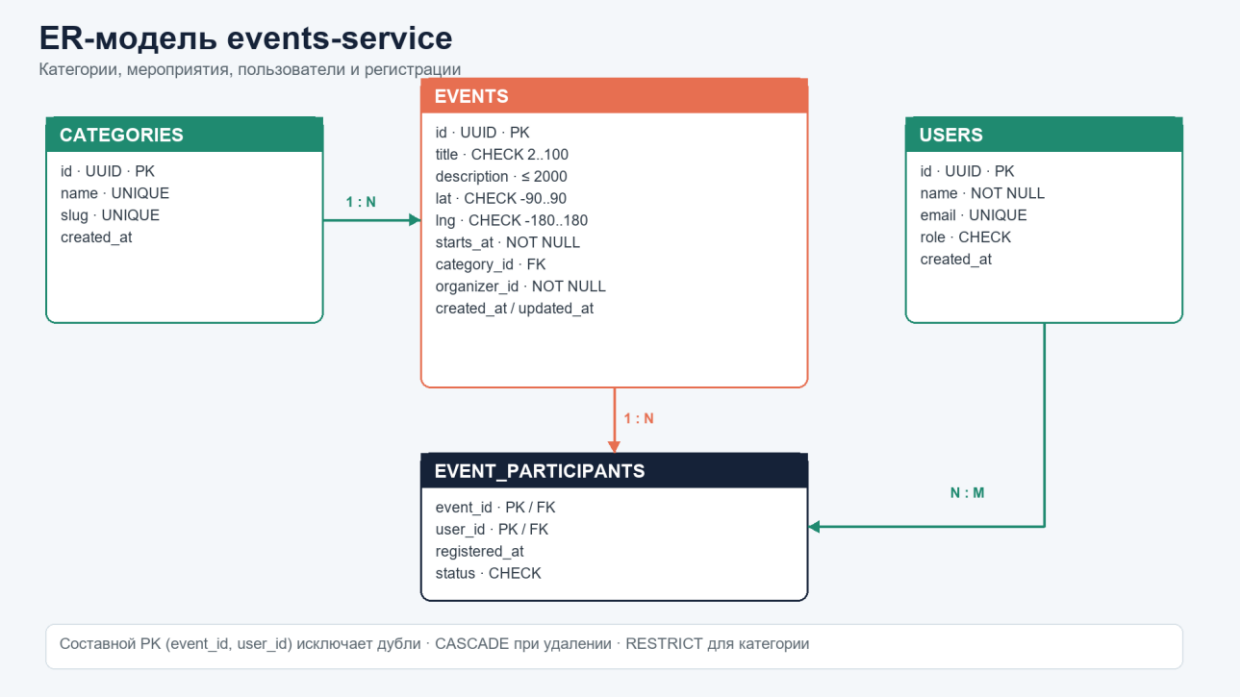


Рисунок 2 — ER-модель events-service

Модель данных включает справочник категорий, мероприятия, пользователей и регистрации. Связь categories → events имеет тип 1:N с ограничением RESTRICT: категорию нельзя удалить, пока в ней есть события. Связь users ↔ events реализована через event_participants с типом N:M и каскадным удалением.

Таблица	Ключевые поля	Ограничения
categories	id, name, slug, created_at	name и slug уникальны
events	id, title, description, lat, lng, starts_at, category_id, organizer_id	CHECK длины, координат; FK на categories
users	id, name, email, role, created_at	email уникален; role ∈ admin/organizer/user
event_participants	event_id, user_id, registered_at, status	составной PK; status ∈ registered/attended/cancelled

Таблица 5 — Основные сущности и ограничения

Составной первичный ключ (event_id, user_id) в event_participants гарантирует, что один пользователь регистрируется на мероприятие только один раз. Внешние ключи event_id и user_id используют ON DELETE CASCADE, а category_id в events — ON DELETE RESTRICT.

7. Миграции, ограничения и индексы

Схема создаётся только миграциями Alembic, а не вручную через SQL-консоль. Это позволяет воспроизвести базу с нуля и продемонстрировать откат.

Миграция	Что делает
0001	Создаёт categories и events
0002	Добавляет categories.slug и составной индекс
0003	Создаёт users и event_participants (N:M)

Таблица 6 — Миграции events-service

Основные индексы и их назначение:

Индекс	Таблица	Колонки	Зачем
ix_events_category_id	events	category_id	фильтр по категории, поддержка FK
ix_events_category_starts_at	events	category_id, starts_at	фильтр категория+даты и сортировка
ix_event_participants_user_id	event_participants	user_id	события пользователя, поддержка FK

Таблица 7 — Индексы и обоснование

Транзакции демонстрируются в verify_tx.py: при ошибке CHECK на событии откатывается и вставка категории; пакетная вставка событий при сбое одной записи откатывается целиком. verify_schema.py проверяет N:M, защиту от дублей и каскадное удаление.

```
cd events-service
alembic upgrade head      # 0001 -> 0002 -> 0003
alembic downgrade -1      # откат последней миграции
alembic upgrade head      # вернуть схему
```

8. Контракт REST API

Контракт описан в events-service/openapi.yaml (OpenAPI 3.0, версия 4.0.0). Для защищённых операций используется BearerAuth. Примеры запросов находятся в events-service/docs/examples.http.

Метод и путь	Доступ	Назначение
POST /auth/register	публично	регистрация user / organizer
POST /auth/login	публично	проверка пароля и выдача JWT
GET /auth/me	JWT	текущий пользователь
GET /users	admin	список пользователей
GET /events	публично	список, фильтры и пагинация
POST /events	organizer / admin	создание события
PUT/PATCH/DELETE /events/{id}	owner / admin	изменение или удаление
POST /attendance	JWT	отметка участия
GET /attendance/event/{id}	публично	статистика события
GET /attendance/user/{id}	self / admin	отметки пользователя

Таблица 8 — Основные эндпоинты

Пример создания события:

```
POST /events
Authorization: Bearer <ORG_TOKEN>
Content-Type: application/json

{
  "title": "Концерт в парке",
  "description": "Живая музыка на открытой площадке",
  "lat": 55.7558,
  "lng": 37.6176,
  "starts_at": "2026-10-01T19:00:00Z",
  "category": "concert"
}

HTTP/1.1 201 CREATED
{"id":"...","title":"Концерт в парке","organizer_id":"..."}
```

HTTP-коды: 200 — успешное чтение/обновление, 201 — создание, 204 — удаление, 400 — валидация, 401 — нет или просрочен токен, 403 — недостаточно прав, 404 — объект не найден, 409 — конфликт уникальности, 500 — внутренняя ошибка.

9. Валидация и единый формат ошибок

Валидация выполняется до обращения к бизнес-логике. Ошибки собираются в details, чтобы клиент мог показать все проблемные поля сразу.

Поле	Правило
title	строка от 2 до 100 символов
description	строка не длиннее 2000 символов
lat / lng	числа в диапазонах -90..90 и -180..180
starts_at	ISO 8601
category	значение из справочника
limit / page	limit 1..100 (по умолчанию 20), page ≥ 1

Таблица 9 — Правила валидации

Единый формат ошибки:

```
HTTP/1.1 400 BAD REQUEST
{
  "error": {
```

```
    "code": "ValidationError",
    "message": "Ошибка валидации",
    "details": [
      {"field": "lat", "message": "должно быть числом"}
    ]
  }
}
```

Один и тот же формат используется в users-service, events-service и attendance-service. Глобальные обработчики не возвращают stack trace и приводят ответы 400/401/403/404/405/409/500 к общей схеме.

10. Пагинация и фильтрация

Список событий поддерживает фильтр по категории и диапазону дат, пагинацию и сортировку по starts_at. Значения limit и page проверяются; limit не может превышать 100.

```
GET /events?category=concert&date_from=2026-10-01
    &date_to=2026-10-31&limit=2&page=1

HTTP/1.1 200 OK
{
  "items": [
    {"id":"...","title":"Концерт в парке",
     "category":"concert","starts_at":"2026-10-01T19:00:00Z"}
  ],
  "page": 1,
  "limit": 2,
  "total": 1
}
```

11. Аутентификация

Выбран JWT (HS256, срок действия 60 минут, issuer event-map-users-service). Пароли хранятся только как Argon2-хеш. При входе users-service проверяет пароль и выдаёт access-токен; остальные сервисы проверяют токен локально общим секретом.

Поток входа:

1. POST /auth/register: валидация логина, email, пароля и роли.
2. Пароль хешируется Argon2; роль ограничена user или organizer.
3. POST /auth/login: проверка пароля, выдача JWT.
4. Клиент передаёт Authorization: Bearer <token>.
5. Сервис проверяет подпись, exp и iss; sub и role попадают в g.current_user.

```
{
  "sub": "user-uuid",
  "role": "organizer",
  "iss": "event-map-users-service",
  "exp": 1760000000,
  "iat": 1759996400
}
```

JWT выбран потому, что сервисы разделены и должны проверять пользователя без обращения к сессионному хранилищу. Ограничение: отзыв токена до истечения срока не поддерживается; refresh-токены и blacklist вынесены в план PR6.

12. Авторизация и роли

Аутентификация (401) и авторизация (403) разделены. Роль проверяется декоратором require_role, а владелец объекта — функцией _authorize_event: organizer_id события сравнивается с sub из JWT. Admin обходит проверку владельца.

Операция	user	organizer	admin
GET /events	да	да	да
POST /events	нет	да	да
PUT/PATCH/DELETE своего события	нет	да	да
PUT/PATCH/DELETE любого события	нет	нет	да
POST /attendance	да	да	да
GET /users	нет	нет	да
GET /attendance/user/{id}	self	self	любой

Таблица 10 — Матрица ролей

organizer_id и user_id никогда не берутся из тела запроса: они определяются по проверенному JWT. Это защищает от массового присвоения ролей и чужих идентификаторов.

13. Безопасность API

Угрозы проанализированы по OWASP API Security Top 10 (2023). В таблице указаны реализованные меры и способ проверки; подробная версия находится в docs/security.md.

Угроза OWASP	Мера	Проверка
API1 BOLA	owner-check + admin override	чужое событие → 403
API2 Broken Auth	Argon2, JWT exp/iss, 401	неверный пароль → 401
API3 Mass Assignment	id и role только из JWT	подмена organizer_id игнорируется
API4 Resource Consumption	limit 1..100, длины полей	limit=1000 → 400
API5 BFLA	require_role, admin-only /users	user → /users 403
API8 Misconfiguration	секреты в .env, DEBUG=0	500 без stack trace

Таблица 11 — Угрозы API и меры защиты

Остаточные риски и план усиления:

- нет rate limiting на /auth/login и публичных GET;
- JWT нельзя отозвать до истечения срока;
- attendance-service хранит данные в памяти;
- проекция users в events-service не синхронизируется автоматически;
- для production нужны HTTPS-терминация и ограничение размера тела запроса.

14. Демонстрация работы

Ниже приведены фрагменты HTTP-ответов для ключевых сценариев: создание события, ошибка валидации, запрет по роли и удаление администратором.

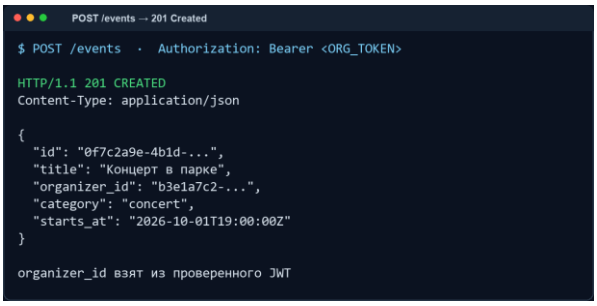


Рисунок 4 — 201 Created: создание события организатором



Рисунок 5 — 400 Bad Request: ошибка валидации



Рисунок 6 — 403 Forbidden: роль user не может удалить событие

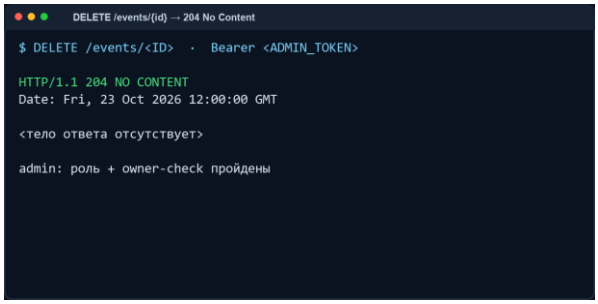


Рисунок 7 — 204 No Content: admin удаляет событие

Результаты проверок: users-service — 10 тестов, events-service — 31 проверка, attendance-service — 10 проверок. Все проверки проходят.

15. Статус проекта и план

Этап	Содержание	Статус
PR1	бизнес-задача, US, архитектура, каркас, /health	выполнено
PR2	OpenAPI, примеры, пагинация, валидация, ошибки	выполнено
PR3	ER, миграции, индексы, N:M, транзакции	выполнено
PR4	JWT, Argon2, роли, owner-check, единые ошибки	выполнено
PR5	attendance в БД, синхронизация users-проекции	план
PR6	rate limiting, refresh-токены, HTTPS	план
PR7	docker-compose, CI, финальная защита	план

Таблица 12 — Статус этапов проекта

Репозиторий: github.com/banumm/event-map-app. Ветка: main. Последний коммит: <вставить SHA>. Ссылка на Kanban: <вставить ссылку>.

Основные риски:

- JWT без отзыва и refresh-токенов;
- attendance-service не персистентен;
- users-проекция в events-service может рассинхронизироваться;
- SQLite подходит для локального MVP, но не для конкурентной нагрузки.

16. Заключение

В рамках СРС №1 спроектирован и реализован MVP Event Map App. Приложение покрывает ключевой сценарий: найти событие на карте, опубликовать мероприятие и отметить участие. Контракт API зафиксирован в OpenAPI, схема БД создаётся миграциями, а аутентификация и авторизация проверяются автотестами.

Безопасность обеспечена Argon2-хешированием паролей, JWT с проверкой подписи/срока/издателя, RBAC и проверкой владельца объекта. Выполнен анализ угроз OWASP API Top 10 и зафиксированы остаточные риски. Дальнейшая работа — persistence для attendance, rate limiting, refresh-токены, CI/CD и production-конфигурация.

17. Список использованных источников

1. OWASP API Security Top 10, 2023. URL: https://owasp.org/API-Security/
2. RFC 7519. JSON Web Token (JWT).
3. RFC 9106. Argon2 Memory-Hard Function for Password Hashing.
4. OpenAPI Specification 3.0.3.
5. Django 5.0 Documentation. URL: https://docs.djangoproject.com/
6. Flask 3.0 Documentation. URL: https://flask.palletsprojects.com/
7. SQLAlchemy 2.0 и Alembic Documentation. URL: https://docs.sqlalchemy.org/
8. Документация проекта: docs/requirements.md, docs/auth.md, docs/auth-check.md, docs/security.md.

Приложение А. Команды запуска и проверки

Users Service (Django):

```
cd users-service
python manage.py migrate
python manage.py create_admin --username admin --email admin@example.com --password
'StrongPass123'
python manage.py runserver 127.0.0.1:8000
```

Events Service (Flask):

```
cd events-service
alembic upgrade head
python app.py
python verify.py
```

Attendance Service (Flask):

```
cd attendance-service
python app.py
python verify.py
```

JWT_SECRET должен совпадать во всех трёх сервисах. Для локального запуска можно использовать значение по умолчанию, для реального окружения — только собственный секрет из .env.